Defensive Publication

Title: A System and Method for Cryptographic Proof of Intent, Geofenced Consent Gating, and Deterministic Hardware Interrupts in Autonomous and Edge-AI Systems

Inventor: Dominique Brack

Publication Date: 12. August 18, 2026

1. Abstract This defensive publication discloses the Refuse Consent Protocol, a multi-layered, machine-readable spatial governance system designed to prevent unauthorized data capture by autonomous agents and edge-AI devices. The protocol utilizes physical and digital spatial markers (Glyphs, 2D Barcodes, URLs, and RFID/NFC) to trigger deterministic hardware or software interrupts, overriding user control to halt audio, visual, or spatial recording.

2. Problem Statement The proliferation of autonomous drones, wearable smart glasses, and robotic agents utilizing Edge AI (on-device processing) allows for real-time spatial data extraction without centralized cloud transmission. Consequently, bystanders, enterprises, and property owners have no effective mechanism to enforce privacy or restrict access. Traditional visual indicators (like 3mm LEDs) are easily circumvented, and digital geofences lack real-world optical "ground truth" and zero-day agility, creating a massive governance void in physical spaces.

3. Background and Known Solutions Existing solutions for spatial restriction rely heavily on centralized, digital "No-Fly Zones" (e.g., DJI Fly Safe) or passive visual indicators (recording LEDs on wearables). These solutions are ineffective because centralized digital geofences are vulnerable to GPS spoofing/jamming, require active internet connectivity to update, and cannot rapidly respond to dynamic, real-world events (e.g., a sudden traffic accident). Passive visual LEDs offer no mechanical enforcement, leaving the burden on the bystander to notice the recording and confront the user, providing zero machine-level governance over the recording device.

4. Novelty Statement This invention introduces a decentralized, physical-to-digital "Ground Truth" protocol that forces immediate, deterministic compliance at the sensor level. By combining adversarial visual markers (Glyphs) that trigger edge-AI hardware interrupts with cryptographically verified "Proof of Intent" (via PKI handshakes, geofenced verification, and air-gapped pre-loaded ledgers), the system shifts consent enforcement from social expectation to an unavoidable, machine-readable physical infrastructure.

5. Implementation A person skilled in the art can implement this protocol to establish spatial governance through the following sequence of steps:

1. **Marker Ingestion:** Configure the optical sensor and computer vision pipeline of an autonomous agent to recognize a specific asymmetrical visual geometry (the Glyph).
2. **Deterministic Interrupt:** Program the agent's edge-AI firmware such that recognition of the Glyph immediately triggers a hardware-level interrupt (e.g., dropping the video frame or halting the microphone payload) prior to data processing or storage.
3. **Cryptographic Handshake (Layer 2+):** For verified infrastructure, the agent reads a secondary marker embedded within the Glyph (e.g., a 2D Barcode/QR code,

dynamic E-Ink display, or RFID). The agent sends a query containing its local telemetry (GNSS data, timestamp) to a Certificate Authority (the Registrar).

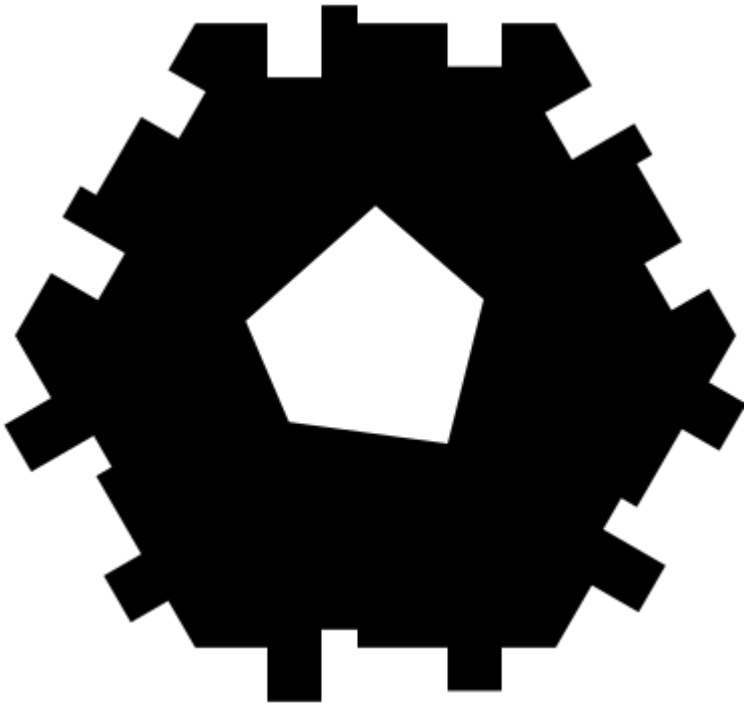
4. **Validation and Audit:** The Registrar returns a PKI-signed JSON payload confirming the restriction parameters (e.g., "Active within a 50m radius of given coordinates"). The agent cross-references this payload with its internal telemetry. If valid, the interrupt holds; if invalid (spoofing detected), the optical command is ignored and an audit log is generated.
5. **Air-Gapped Sync:** For offline environments, the agent downloads the Registrar's cryptographic ledger (e.g., a Merkle tree) pre-flight and performs Step 4 entirely on local flash memory without requiring an active internet connection.

6. Embodiment of Example This novel protocol can be implemented in at least the following three ways:

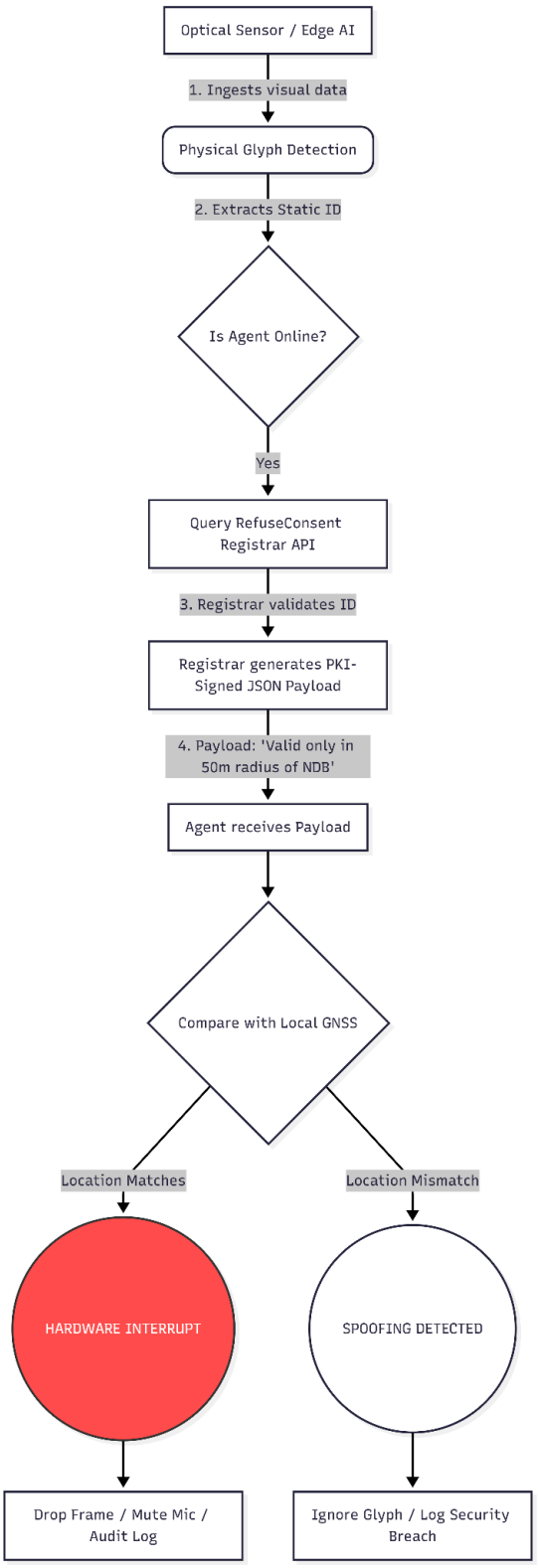
- **Example 1: The Public "Opt-Out" (Hardware Interrupt):** A pedestrian wears a 3D-printed Glyph on their backpack. As a wearer of smart glasses looks at them, the glasses' on-board AI detects the Glyph in the frame, triggering an immediate hardware interrupt that blurs the pedestrian or drops the recording frame entirely, enforcing an optical Opt-Out.
- **Example 2: The Enclosed "Opt-In" (Gatekeeper Mechanism):** An enterprise boardroom defaults to a "Zero Trust" recording state. Before entering, a participant's AI wearable scans the room's RFID marker. The device actively transmits a cryptographic "Consent to Not Record" to the room's gatekeeper system. Access to the room (or digital meeting) is only granted once all active, PKI-signed consents are registered, preventing corporate espionage.
- **Example 3: Dynamic Incident Zones (Zero-Day Agility):** Emergency responders at a traffic accident deploy police tape printed with the Glyph. Passing autonomous delivery drones and vehicles optically ingest the Glyph, which overrides their default navigation and recording systems, forcing them to instantly reroute and disable data capture around the crash site without waiting for a centralized database update.

7. Supporting Images/Figures

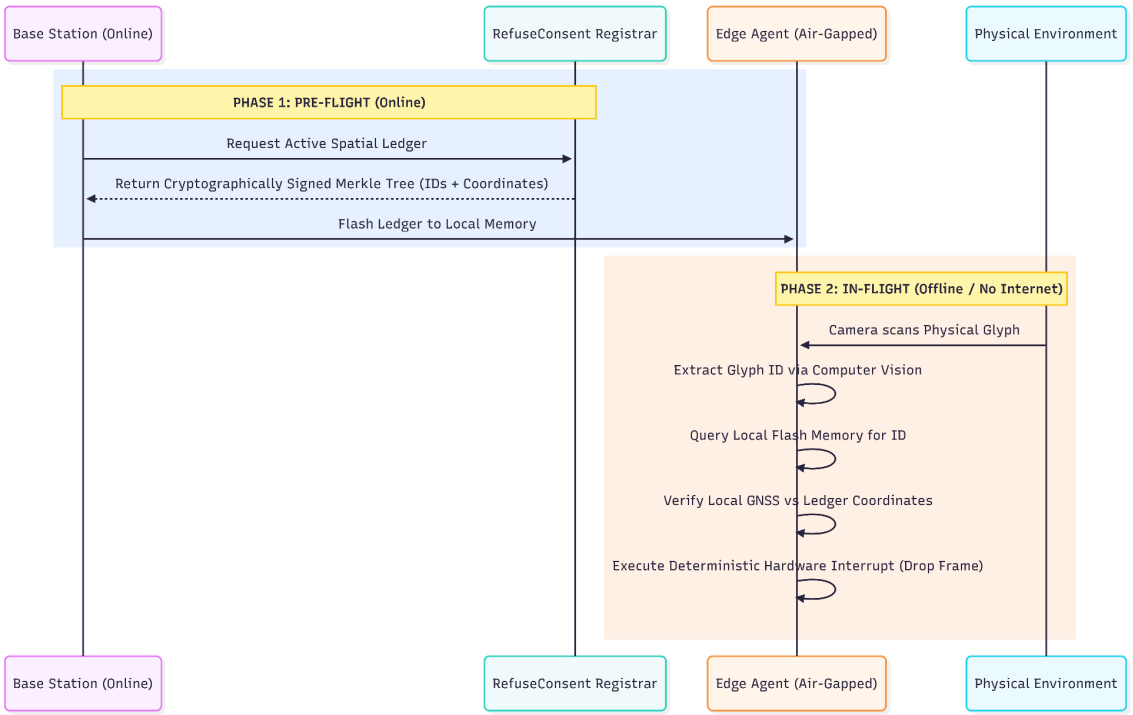
- **[Figure 1]** 3D CAD schematic of the asymmetrical Refuse Consent Glyph demonstrating the optical geometries required for computer vision ingestion.



- **[Figure 2]** Architectural flowchart detailing the Cryptographic Proof of Intent handshake and Geofenced Verification loop between the autonomous agent and the Registrar.



- **[Figure 3]** System diagram of the Air-Gapped Pre-Loaded Sync architecture for offline UAV operations.



8. Appendix A: Programmatic Geometry and Architecture Generation

A.1. Glyph Generative Algorithm (Python)

```
import math
import os
import webbrowser

def generate_single_glitch_hexagon():
    # --- Geometrie Parameter ---
    R = 180
    sqrt3 = math.sqrt(3)

    vertices = [
        (-R, 0), (-R/2, -sqrt3/2 * R), (R/2, -sqrt3/2 * R),
        (R, 0), (R/2, sqrt3/2 * R), (-R/2, sqrt3/2 * R)
    ]

    profile = [
        (0.0, 0.0), (0.2, 0.0), (0.2, 0.15), (0.35, 0.15),
        (0.35, -0.05), (0.45, -0.05), (0.45, 0.0), (0.7, 0.0),
        (0.7, 0.12), (0.85, 0.12), (0.85, 0.0), (1.0, 0.0)
    ]
    def get_edge_points(v_start, v_end, is_inverse):
        dx, dy = v_end[0] - v_start[0], v_end[1] - v_start[1]
        nx, ny = -dy, dx
        pts = []
        for t, offset in profile:
            _t, _offset = (1.0 - t, -offset) if is_inverse else (t, offset)
            pts.append((v_start[0] + dx * _t + nx * _offset, v_start[1] + dy * _t + ny * _offset))
        if is_inverse: pts.reverse()
        return pts

    # --- SVG Pfad generieren ---
    path_data = []
    for i in range(6):
        pts = get_edge_points(vertices[i], vertices[(i+1)%6], i >= 3)
        if i == 0: path_data.append(f"M {pts[0][0]:.3f} {pts[0][1]:.3f}")
        for p in pts[1:]: path_data.append(f"L {p[0]:.3f} {p[1]:.3f}")
    path_data.append("Z")
    scale = 3.6
      hole_vertices = [(0, -18*scale), (15*scale, -5*scale), (10*scale, 15*scale), (-12*scale,
12*scale), (-18*scale, -2*scale)]
    path_data.append(f"M {hole_vertices[0][0]:.3f} {hole_vertices[0][1]:.3f}")
    for p in hole_vertices[1:]: path_data.append(f"L {p[0]:.3f} {p[1]:.3f}")
    path_data.append("Z")

    svg_path = " ".join(path_data)
```

```
    # --- SVG Datei zusammenbauen (mit offiziellem XML Header) ---
    svg = []
    svg.append('<?xml version="1.0" encoding="UTF-8"?>')
            svg.append('<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 500 500"
style="background-color: white;">')
                        svg.append(f'<path d="{svg_path}" fill="black" fill-rule="evenodd"
transform="translate(250, 250)" />')
    svg.append('</svg>')

    # --- Datei speichern (Zwingend als UTF-8) ---
    current_dir = os.path.dirname(os.path.abspath(__file__))
    file_path = os.path.join(current_dir, "refuse_consent_logo.svg")

    with open(file_path, "w", encoding="utf-8") as f:
        f.write("\n".join(svg))

    print(f"ERFOLG! Einzel-Logo wurde hier gespeichert:\n{file_path}\n")
    return file_path

if __name__ == "__main__":
    try:
        print("Generiere Einzel-Glitch-Hexagon...")
        saved_file_path = generate_single_glitch_hexagon()

        print("Versuche Browser zu öffnen...")
        webbrowser.open(f"file://{saved_file_path}")

    except Exception as e:
        print(f"\nFEHLER: {e}")

    input("\nDrücke Enter, um das Fenster zu schliessen...")
```

A.2. Architectural Handshake Flowchart (Mermaid)

```
graph TD
    A[Optical Sensor / Edge AI] -->|1. Ingests visual data| B(Physical Glyph Detection)
    B -->|2. Extracts Static ID| C{Is Agent Online?}

    C -->|Yes| D[Query RefuseConsent Registrar API]
    D -->|3. Registrar validates ID| E[Registrar generates PKI-Signed JSON Payload]
    E -->|4. Payload: 'Valid only in 50m radius of NDB'| F[Agent receives Payload]

    F --> G{Compare with Local GNSS}
    G -->|Location Matches| H((HARDWARE INTERRUPT))
    H --> I[Drop Frame / Mute Mic / Audit Log]

    G -->|Location Mismatch| J((SPOOFING DETECTED))
    J --> K[Ignore Glyph / Log Security Breach]

    classDef interrupt fill:#ff4d4d,stroke:#333,stroke-width:2px,color:#fff;
    class H interrupt;
    class J fill:#ff9900,stroke:#333,stroke-width:2px;
```

A.3. Air-Gapped Sync Architecture (Mermaid)

```
sequenceDiagram
    participant Depot as Base Station (Online)
    participant Registrar as RefuseConsent Registrar
    participant Drone as Edge Agent (Air-Gapped)
    participant World as Physical Environment

    rect rgb(230, 240, 255)
    Note over Depot, Registrar: PHASE 1: PRE-FLIGHT (Online)
    Depot->>Registrar: Request Active Spatial Ledger
    Registrar-->>Depot: Return Cryptographically Signed Merkle Tree (IDs + Coordinates)
    Depot->>Drone: Flash Ledger to Local Memory
    end
    rect rgb(255, 240, 230)
    Note over Drone, World: PHASE 2: IN-FLIGHT (Offline / No Internet)
    World->>Drone: Camera scans Physical Glyph
    Drone->>Drone: Extract Glyph ID via Computer Vision
    Drone->>Drone: Query Local Flash Memory for ID
    Drone->>Drone: Verify Local GNSS vs Ledger Coordinates
    Drone->>Drone: Execute Deterministic Hardware Interrupt (Drop Frame)
    end
```